## Garph

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': ['H'],
    'E': [],
    'F': [],
    'G': ['P'],
    'H': [],
    'P': []
}
```

**Question 1:**

```
def BFS(graph, start, goal):
    visited = []
    queue = [[start]]

    while queue:
        path = queue.pop(0)
        node = path[-1]

        if node not in visited:
            visited.append(node)

            if node == goal:
                print("BFS Transition Path:", visited)
                return path

            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                queue.append(new_path)
def DFS(graph, start, goal):
    visited = []
    stack = [[start]]

    while stack:
        path = stack.pop()
        node = path[-1]

        if node not in visited:
            visited.append(node)

            if node == goal:
                print("DFS Transition Path:", visited)
                return path

            for neighbor in graph[node]:
                new_path = list(path)
                new_path.append(neighbor)
                stack.append(new_path)
bfs_path = BFS(graph, 'A', 'P')
print("BFS Final Path:", bfs_path)

dfs_path = DFS(graph, 'A', 'P')
print("DFS Final Path:", dfs_path)
```

**Output (Question 1):**

```
BFS Transition Path: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'P']
BFS Final Path: ['A', 'C', 'G', 'P']
DFS Transition Path: ['A', 'C', 'G', 'P']
DFS Final Path: ['A', 'C', 'G', 'P']
```

**Question 2:**

```python
def second_largest(arr):
    arr = list(set(arr))

    if len(arr) < 2:
        return None

    arr.sort()
    return arr[-2]

print("Second Largest:", second_largest([10, 20, 5, 20, 8]))
print("Second Largest:", second_largest([5, 5, 5]))
```

**Output (Question 2):**

```
Second Largest: 10
Second Largest: None
```

**Question 3:**

```
def DLS(graph, node, goal, limit):

    if node == goal:
        return True

    if limit == 0:
        return False

    for neighbor in graph[node]:
        if DLS(graph, neighbor, goal, limit - 1):
            return True

    return False

print("Depth 3:", DLS(graph, 'A', 'P', 3))
print("Depth 2:", DLS(graph, 'A', 'P', 2))
```

**Output (Question 3):**

```
Depth 3: True
Depth 2: False
```